DAY 2

Python Programming Fundamentals

Detailed beginner notes with code examples, outputs, and practice programs

In this PDF:

- Python introduction and setup
- Variables and data types
- Input/output
- Operators
- Conditions and loops
- Step-by-step logic
- Six required practical programs
- Additional examples, debugging tips, exercises, and revision

1. Introduction to Python

Python is a high-level, general-purpose programming language with readable syntax. It is used for web development, automation, data analysis, AI, testing, and scripting.

Example 1: First program

```
print("Hello, World!")
print("Welcome to Python")
```

Output

```
Hello, World!
Welcome to Python
```

Explanation: print() displays text or values on the screen. Text is written inside quotation marks.

2. Installing Python and Running Code

Check whether Python is installed in Windows PowerShell or the VS Code terminal:

```
python --version
python -m pip --version
```

If a version appears, Python is available from that terminal. If Python is not recognised, install it from the official website: python.org/downloads. During installation, enable the option to add Python to PATH.

Create and run a .py file

In VS Code, create a file named `day2.py`, write code, save it, and run:

```
python day2.py
```

Optional: virtual environment

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

The basic programs in this guide do not require extra packages.

3. Variables and Data Types

A variable is a name that refers to a value. The equals sign (=) assigns a value.

Example 1: Variables

```
name = "Shweta"
age = 21
percentage = 88.4
is_student = True

print(name)
print(age)
print(percentage)
print(is_student)
```

Output

```
Shweta
21
88.4
True
```

Example 2: Check data type

```
number = 25
print(type(number))

price = 12.5
print(type(price))
```

Output

```
<class 'int'>
<class 'float'>
```

Common data types

Type	Example	Meaning
str	"Hello"	Text
int	10	Whole number
float	10.5	Decimal number
bool	True / False	Logical value
list	[1, 2, 3]	Ordered, changeable collection
tuple	(1, 2, 3)	Ordered, unchangeable collection
dict	{"city": "Pune"}	Key-value pairs
set	{1, 2, 3}	Collection of unique values

Example 3: List, tuple, dictionary, and set

```
numbers = [10, 20, 30]
point = (4, 5)
student = {"name": "Shweta", "city": "Pune"}
unique_numbers = {1, 2, 2, 3}

print(numbers)
print(point)
print(student["name"])
print(unique_numbers)
```

Note: a set stores unique values. The order in which set items display is not guaranteed.

4. Input and Output

input() reads what the user types and returns text. Use int() for whole numbers or float() for decimal numbers.

Example 1: Read a name

```
name = input("Enter your name: ")
print("Hello", name)
```

Sample run:

```
Enter your name: Shweta
Hello Shweta
```

Example 2: Add two numbers

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
total = a + b
print("Sum =", total)
```

Sample run:

```
Enter first number: 10
Enter second number: 20
Sum = 30
```

5. Operators

Operators perform calculations, comparisons, or logical checks.

Arithmetic operators

```
a = 10
b = 3
print(a + b)   # 13: addition
print(a - b)   # 7: subtraction
print(a * b)   # 30: multiplication
print(a / b)   # 3.333...: division
```

```
print(a // b)  # 3: floor division
print(a % b)   # 1: remainder
print(a ** b)  # 1000: power
```

Comparison and logical operators

```
x = 5
y = 3
print(x > y)                # True
print(x == y)               # False
print(x != y)               # True
print(x > 0 and y > 0)      # True
print(x < 0 or y > 0)       # True
print(not (x > y))          # False
```

Comparison operators return True or False. Logical operators combine or reverse conditions.

Assignment and membership

```
count = 5
count += 2
print(count)            # 7
print("a" in "cat")     # True
print("z" not in "cat")  # True
```

6. Conditional Statements

Use if, elif, and else to choose a block of code. Python uses indentation (usually four spaces) to group statements.

Example 1: Check pass or fail

```
marks = int(input("Enter marks: "))

if marks >= 35:
    print("Pass")
else:
    print("Fail")
```

Sample output

```
Enter marks: 72
Pass
```

Example 2: Grade using if/elif/else

```
marks = int(input("Enter marks: "))

if marks >= 75:
    print("Distinction")
elif marks >= 60:
    print("First class")
elif marks >= 35:
    print("Pass")
else:
    print("Fail")
```

Test the program with 80, 65, 40, and 20.

Example 3: Check positive, negative, or zero

```
number = int(input("Enter a number: "))

if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

7. Loops

A loop repeats code. range(start, stop) includes start but excludes stop.

Example 1: for loop

```
for i in range(1, 6):
    print(i)
```

Output: 1, 2, 3, 4, 5, each on a separate line.

Example 2: while loop

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

The counter must change so the loop eventually stops.

Example 3: break and continue

```
for n in range(1, 6):
    if n == 3:
        continue  # skip 3
    if n == 5:
        break     # stop the loop
    print(n)

# Output: 1, 2, 4
```

8. Basic Problem-Solving

Use this simple method for programming questions:

• Understand the required input and output.

• Write the steps in plain language (algorithm).

• Choose variables, conditions, and loops.

• Write the code with correct indentation.

• Test normal values and edge cases.

• Read errors and correct the problem.

Example algorithm: even or odd

Step 1: Read a number. Step 2: Find number % 2. Step 3: If the remainder is zero, it is even; otherwise, it is odd.

9. Practical 1 — Even or Odd

Logic: an even integer has remainder 0 when divided by 2.

```
number = int(input("Enter a number: "))

if number % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

Sample output

```
Enter a number: 8
Even number
```

Try: 0, 7, and -4.

10. Practical 2 — Largest of Three Numbers

Logic: compare each number with the other two. The >= operator also handles equal values.

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
```

```
    largest = b
else:
    largest = c

print("Largest number is:", largest)
```

Sample output

```
Enter first number: 12
Enter second number: 7
Enter third number: 19
Largest number is: 19.0
```

11. Practical 3 — Factorial

Factorial of n is the product of integers from 1 to n. By definition, 0! = 1. Factorial is not defined for negative integers.

```
n = int(input("Enter a non-negative integer: "))

if n < 0:
    print("Factorial is not defined for negative integers")
else:
    factorial = 1
    for i in range(1, n + 1):
        factorial = factorial * i
    print("Factorial is:", factorial)
```

Sample output

```
Enter a non-negative integer: 5
Factorial is: 120
```

Calculation for 5: 1 × 2 × 3 × 4 × 5 = 120.

12. Practical 4 — Prime Number

A prime number is greater than 1 and has exactly two positive divisors: 1 and itself.

```
n = int(input("Enter an integer: "))

if n <= 1:
    print("Not a prime number")
else:
    is_prime = True
    divisor = 2

    while divisor * divisor <= n:
        if n % divisor == 0:
            is_prime = False
            break
        divisor += 1

    if is_prime:
        print("Prime number")
    else:
        print("Not a prime number")
```

Sample output

```
Enter an integer: 17
Prime number
```

Try 2, 9, 1, 0, and -5.

13. Practical 5 — Reverse a Number

Use % 10 to get the last digit and // 10 to remove the last digit. This program preserves a negative sign.

```
n = int(input("Enter an integer: "))

sign = -1 if n < 0 else 1
n = abs(n)
reverse = 0
```

```
while n > 0:
    digit = n % 10
    reverse = reverse * 10 + digit
    n //= 10

reverse *= sign
print("Reversed number is:", reverse)
```

Sample output

```
Enter an integer: 1234
Reversed number is: 4321
```

For input 1200, the numeric result is 21 because integer values do not preserve leading zeroes.

14. Practical 6 — Multiplication Table

```
n = int(input("Enter a number: "))

for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

Sample output

```
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

15. Extra Example — Sum of First n Natural Numbers

```
n = int(input("Enter a positive integer: "))
total = 0

for i in range(1, n + 1):
    total += i

print("Sum =", total)
```

Example: for n = 5, output is Sum = 15 (1 + 2 + 3 + 4 + 5).

16. Common Errors

SyntaxError: Check brackets, quotes, colons, spelling, and punctuation.

IndentationError: Indent statements consistently, usually with four spaces.

ValueError: Check that numeric input is valid before converting with int() or float().

NameError: Check spelling and make sure the variable is assigned before use.

Infinite loop: Check that the while-loop condition can eventually become false.

Example of input conversion

```
# This works only when the user enters a whole number
age = int(input("Enter your age: "))
print("Age =", age)
```

17. Practice Exercises

• Write a program to calculate the average of three marks.

• Write a program to count the digits in an integer.

• Write a program to find the sum of digits of a number.

• Print all even numbers from 1 to 50.

• Check whether a year is a leap year.

• Find the smallest of three numbers.

Try to solve these exercises yourself before looking for a solution.

18. Quick Revision and Completion Checklist

• print() displays output; input() reads text from the user.

• Use int() or float() to convert numeric input.

• Variables store values; common types are str, int, float, and bool.

• Arithmetic operators include +, -, *, /, //, %, and **.

• if/elif/else make decisions; indentation defines blocks.

• for and while loops repeat code; break exits and continue skips an iteration.

• Test normal inputs and edge cases.

■ Python environment configured

■ Variables and data types practised

■ Operators practised

■ Conditions and loops practised

■ Even/odd

■ Largest of three

■ Factorial

■ Prime number

■ Reverse number

■ Multiplication table

End of Day 2 notes. Save and run each program in a separate .py file if required by your instructor.